

SERVICE ACTUEL · AI-GEN-ASSISTANT

Assistants, contextes et corpus RAG

6 capacités analysées

services/ai-gen-assistant · Python/FastAPI, PostgreSQL/SQLAlchemy/Alembic, Milvus, S3, Kafka et schedulers ; dépendances Python de génération.

Configurer les assistants et leurs contextes, converser avec outils et sources, ingérer les corpus et gérer les artefacts générés.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : AI, Embeddings, MCP, Flows, Writer, Publisher, Files, IAM, Audit, Runtimes.

Périmètre de lecture

- Analyses existantes 11-ml-embeddings et 14-front-api reprises comme contexte ; vérifications ciblées des routes, outils et implémentations citées ci-dessous.
- Instructions racine et AGENTS.md du composant lorsqu'il existe ; aucun binaire propriétaire inspecté.
- Lecture statique ciblée pour finalisation ; les branches non exécutées et le déploiement restent non qualifiés.

Objets et cycle de vie

- Contextes et versions ; conversations et messages.
- Corpus, sources, documents, métadonnées, chunks et transactions.
- Toolboxes, variables utilisateur, sessions de fichiers, templates et artefacts.

Dépendances du service

- IAM et outbox de droits ; proxy LLM/embeddings ; data-collect ; MCP ; PostgreSQL/S3/Milvus/Kafka.

Fonctionnalités et compatibilité à préserver

AIGA-01 Cible non qualifiée

Configurer un contexte d'assistant, ses ressources et son partage

COMPORTEMENT OBSERVÉ

Le contexte est rattaché à un projet et porte nom, description, instructions, personnalisation, ressources corpus/datasets/fichiers, toolboxes et choix de modèle avec réglages de génération. Les routes créent, listent et modifient ces configurations, leur statut et leur propriétaire. La création résout le miroir utilisateur avant écriture et canonicalise le projet. Des routes distinctes gèrent le partage. Les faits de création, propriété et partage passent par l'outbox IAM lorsque cette émission est activée.

UTILISATEURS ET RÔLES

Auteurs d’assistants, membres de projets et propriétaires.

INTERFACES DANS LE CODE

ContextRouter ; POST /projects/{project_id}/contexts ; ContextShareRouter.

DONNÉES ET ÉTATS

Context, ContextOwner, ConversationResources, références de sessions de fichiers, miroirs utilisateur/projet.

DÉPENDANCES ET ÉCHANGES

IAM, PostgreSQL, modèles du proxy, événements de droits.

COMPORTEMENT CIBLE PROPOSÉ

AI Rust gère les configurations versionnées et les mandats de ressources ; IAM décide partage et propriété. Les ressources référencent des éditions autorisées.

MIGRATION DE CETTE CAPACITÉ

Importer UUID, propriétaires et références, reconstruire les faits IAM puis comparer les accès avant bascule.

CRITÈRE D’ACCEPTATION À EXÉCUTER

Créer un contexte avec trois types de ressource, le partager puis révoquer ; les autres projets ne découvrent aucune ressource interdite.

Références de l’analyse

[monorepo · services/ai-gen-assistant/app/src/context/application/router/context_router.py:103](#) Création et modèle de configuration.

[monorepo · services/ai-gen-assistant/AGENTS.md:49](#) Faits IAM et outbox transactionnelle.

AIGA-02 **Cible non qualifiée**

Historiser, comparer et publier les versions de contexte

COMPORTEMENT OBSERVÉ

Le service expose liste et détail des versions, diff, duplication, publication d’une version et suppression simple ou multiple. Une lecture peut résoudre la configuration effective d’une version sans muter le contexte courant. La publication citée ici concerne la configuration de l’assistant ; elle ne démontre aucune publication de données Iceberg.

UTILISATEURS ET RÔLES

Auteurs et responsables de contextes.

INTERFACES DANS LE CODE

ContextVersionRouter : list_context_versions, diff_context_version, duplicate_context_version, publish_context_version.

DONNÉES ET ÉTATS

Versions de contexte et configuration effective ; pagination et auteurs résolus.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, service de versions et miroir d’identité.

COMPORTEMENT CIBLE PROPOSÉ

AI conserve une version de plan immuable ; Publisher reste seul propriétaire des publications de données déclenchées par un plan.

MIGRATION DE CETTE CAPACITÉ

Reprendre versions et diffs ; conserver une distinction de contrat et de vocabulaire entre configuration publiée et édition de données approuvée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Lire une ancienne version sans changer la tête ; publier une configuration ne rend aucun candidat de données visible.

Références de l'analyse

[monorepo · services/ai-gen-assistant/app/src/context/application/router/context_version_router.py:148](#) Comparaison des versions.

[monorepo · services/ai-gen-assistant/app/src/context/application/router/context_router.py:65](#) Résolution explicite sans mutation.

AIGA-03 **Cible non qualifiée**

Ouvrir et reprendre les conversations avec streaming

COMPORTEMENT OBSERVÉ

Les conversations existent pour un assistant et pour un contexte. Les routes listent et consultent les messages, créent un tour, reprennent un échange, modifient les propriétés ou suppriment la conversation. Les créations et reprises utilisent une réponse en streaming ; l'absence de modèle instruct par défaut a une réponse dédiée. Une API publique de contextes expose aussi ces parcours avec une dépendance de contrôle d'accès au contexte. Les artefacts graphiques générés possèdent leur propre téléchargement.

UTILISATEURS ET RÔLES

Utilisateurs de chat, applications clientes et assistants intégrés.

INTERFACES DANS LE CODE

ConversationRouter ; ContextConversationRouter ; PublicContextRouter /v1/contexts/{context_id}.

DONNÉES ET ÉTATS

Conversations, messages, ressources de conversation et artefacts ; état en PostgreSQL/S3 selon l'objet.

DÉPENDANCES ET ÉCHANGES

AI proxy, IAM, toolboxes/MCP, stockage d'artefacts.

COMPORTEMENT CIBLE PROPOSÉ

AI Rust orchestre des tours bornés, annulation et budgets ; messages et citations portent la version de contexte et les éditions effectivement lues.

MIGRATION DE CETTE CAPACITÉ

Migrer historiques et identifiants ; qualifier la compatibilité des événements streaming et les reprises avant bascule des clients.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une interruption reprend selon un identifiant de tour sans lancer deux effets ; une révocation du contexte bloque lecture et nouveau tour.

Références de l'analyse

[monorepo · services/ai-gen-assistant/app/src/assistant/application/router/conversation_router.py:207](#) Création et streaming.

[monorepo](#) · [services/ai-gen-assistant/app/src/public_api/public_context_router.py:129](#) Dépendance commune d'accès au contexte public.

AIGA-04 Cible non qualifiée

Définir les corpus et exécuter leurs ingestions documentaires

COMPORTEMENT OBSERVÉ

Les corpus sont créés et listés dans un projet, avec recherche d'existence, documents, métadonnées et configuration de refresh. Les transactions corpus/source/document suivent l'ingestion et la suppression. La boucle source utilise une ligne source_transaction_progress créée avec la transaction, revendiquée sous bail et renouvelée ; un autre worker peut reprendre depuis le curseur validé après disparition du propriétaire. La concurrence des workers est configurée dans le scheduler. Les documents possèdent aussi métadonnées et états de transaction consultables.

UTILISATEURS ET RÔLES

Gestionnaires de connaissances, auteurs de corpus et opérateurs.

INTERFACES DANS LE CODE

CorpusRouter ; DocumentRouter ; CorpusTransactionRouter ; SourceTransactionLoopScheduler.

DONNÉES ET ÉTATS

Corpus, Source, Document, transactions et curseurs de progression PostgreSQL.

DÉPENDANCES ET ÉCHANGES

Data-collect pour les sources, stockage objet, extraction de documents, modèle d'embedding.

COMPORTEMENT CIBLE PROPOSÉ

Flows/AI/Embeddings utilisent des runs bornés. Writer prépare un FILESET et des EMBEDDINGS immuables ; Publisher publie sous WAP avant acquittement de la plage source.

MIGRATION DE CETTE CAPACITÉ

Importer les documents et leurs provenances comme éditions auditées ; reprendre les travaux en attente avec correspondance explicite entre ancien curseur et plage source.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Tuer un worker pendant une tranche ; reprendre sans perdre les documents et ne faire avancer le checkpoint publié qu'après publication approuvée.

Références de l'analyse

[monorepo](#) · [services/ai-gen-assistant/app/src/corpus/application/router/corpus_router.py:111](#) Surface de corpus et contrôle projet.

[monorepo](#) · [services/ai-gen-assistant/app/src/corpus/application/scheduler/source_transaction_loop_scheduler.py:57](#) Concurrency, bail et reprise de source.

AIGA-05 Cible non qualifiée

Indexer les chunks et rechercher les corpus autorisés

COMPORTEMENT OBSERVÉ

La recherche publique liste les corpus accessibles puis recherche des documents. Le stockage de chunks repose sur Milvus. Le remplacement écrit les nouveaux chunks puis retire les anciens ; le commentaire et le code indiquent l'absence de transaction Milvus, avec une période possible de duplication. En perte de propriété, le nettoyage cible les identifiants écrits par ce worker. La dimension des vecteurs est actuellement sondée par une requête au modèle.

UTILISATEURS ET RÔLES

Utilisateurs RAG, assistants et MCP corpus.

INTERFACES DANS LE CODE

PublicCorpusRouter list_corpora/search_corpora ; ChunkAdapter.replace_document_chunks.

DONNÉES ET ÉTATS

Collections Milvus, documents, IDs de chunks et watermark de génération.

DÉPENDANCES ET ÉCHANGES

Milvus, proxy embeddings, corpus en PostgreSQL, autorisation IAM.

COMPORTEMENT CIBLE PROPOSÉ

EMBEDDINGS Iceberg conserve vecteurs, hash du contenu, édition source, découpage, modèle et dimension. L'index Milvus ou son remplaçant devient une projection reconstruisible par édition approuvée.

MIGRATION DE CETTE CAPACITÉ

Écrire une génération candidate, auditer dimensions et droits, publier puis construire l'index privé avant bascule de lecture ; ne pas assimiler l'alias d'index à la vérité publiée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Changer de modèle/dimension, perdre l'index et le reconstruire ; aucune recherche ne mélange deux éditions ni ne restitue un document révoqué.

Références de l'analyse

[monorepo · services/ai-gen-assistant/app/src/corpus/infra/adapter/storage/chunk_adapter.py:308](#) Ordre écriture/nettoyage et absence de transaction.

[monorepo · services/ai-gen-assistant/app/src/corpus/infra/adapter/storage/chunk_adapter.py:362](#) Sondage de dimension.

[monorepo · services/ai-gen-assistant/app/src/public_api/public_corpus_router.py:61](#) Recherche publique.

AIGA-06 **Cible non qualifiée**

Outils des assistants et gérer fichiers, modèles de documents et exports

COMPORTEMENT OBSERVÉ

Les toolboxes exposent leurs outils et variables utilisateur, ainsi que leur instance. Le service gère des sessions S3 pour upload/suppression de fichiers. Il reçoit des modèles de documents, les liste, les prévisualise, les modifie et les archive. Les documents générés proposent consultation, formats exportables et téléchargement ; les graphiques ont un export dédié. Des outils de lecture de fichiers, génération de documents/PPTX, graphiques et exécution Python sont présents ; cette lecture n'établit pas à elle seule leur isolation runtime.

UTILISATEURS ET RÔLES

Auteurs d'assistants, utilisateurs de pièces jointes et administrateurs d'outils.

INTERFACES DANS LE CODE

ToolboxRouter ; S3FileSessionRouter ; DocumentTemplateRouter ; GeneratedDocumentRouter ;
GeneratedChartRouter.

DONNÉES ET ÉTATS

Toolbox/Tool, variables par utilisateur, sessions de fichiers, modèles, documents et graphiques générés.

DÉPENDANCES ET ÉCHANGES

MCP, S3, moteur de génération et runtime Python.

COMPORTEMENT CIBLE PROPOSÉ

AI/Runtimes exécutent les outils sous mandat et budgets ; Files contrôle les pièces jointes. Un EXPORT est un effet à reçu, un artefact durable peut être un FILESET publié via Writer/Publisher.

MIGRATION DE CETTE CAPACITÉ

Migrer métadonnées et objets avec hashes ; qualifier isolation Python, droits d'outils, formats et suppression des artefacts avant transfert.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un outil n'accède qu'aux fichiers du mandat ; le même export rejoué retrouve son reçu ; une suppression retire objet, index et liens conservés selon politique.

Références de l'analyse

[monorepo · services/ai-gen-assistant/app/src/assistant/application/router/toolbox_router.py:211](#) Variables de toolbox utilisateur.

[monorepo · services/ai-gen-assistant/app/src/assistant/application/router/document_template_router.py:47](#) Cycle de vie des templates.

[monorepo · services/ai-gen-assistant/app/src/assistant/application/router/generated_document_router.py:115](#) Formats et téléchargement.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- AI Rust gère les configurations versionnées et les mandats de ressources ; IAM décide partage et propriété. Les ressources référencent des éditions autorisées.
- AI conserve une version de plan immuable ; Publisher reste seul propriétaire des publications de données déclenchées par un plan.
- AI Rust orchestre des tours bornés, annulation et budgets ; messages et citations portent la version de contexte et les éditions effectivement lues.
- Flows/AI/Embeddings utilisent des runs bornés. Writer prépare un FILESET et des EMBEDDINGS immuables ; Publisher publie sous WAP avant acquittement de la page source.
- EMBEDDINGS Iceberg conserve vecteurs, hash du contenu, édition source, découpage, modèle et dimension. L'index Milvus ou son remplaçant devient une projection reconstruisible par édition approuvée.
- AI/Runtimes exécutent les outils sous mandat et budgets ; Files contrôle les pièces jointes. Un EXPORT est un effet à reçu, un artefact durable peut être un FILESET publié via Writer/Publisher.

Plan de données proposé

- Ingestion et embedding bornés sous Writer ; recherche sur index construit depuis une édition approuvée ; conversation avec tours et outils bornés.

Découpage du core Rust

- ai-contexts, ai-conversations, ai-tools, corpus-runs et projections embeddings ; calcul Python utilisateur isolé derrière Runtimes.

Identité, autorisation et isolation

- Faits IAM émis sous indicateur ; sujets historiques compatibles Keycloak à migrer.
- Les documents externes et résultats d'outils sont des données non fiables ; jamais une autorité pour élargir le mandat.
- Un index ne doit pas exposer textes ou nombres de candidats interdits.

Exploitation et reprise

- Migrations séparées du démarrage web, hook de préparation et mode migrate ; Alembic hors enforcement migrations racine.
- Boucles sources/documents, leases, outbox et replay à reprendre ; tests Docker non exécutés.
- AGENTS signale un décalage du paquet cleyrop_common publié et des imports monorepo ; ne pas présumer le démarrage qualifié.

Séquence de migration

- Importer UUID, propriétaires et références, reconstruire les faits IAM puis comparer les accès avant bascule.
- Reprendre versions et diffs ; conserver une distinction de contrat et de vocabulaire entre configuration publiée et édition de données approuvée.
- Migrer historiques et identifiants ; qualifier la compatibilité des événements streaming et les reprises avant bascule des clients.
- Importer les documents et leurs provenances comme éditions auditées ; reprendre les travaux en attente avec correspondance explicite entre ancien curseur et plage source.
- Écrire une génération candidate, auditer dimensions et droits, publier puis construire l'index privé avant bascule de lecture ; ne pas assimiler l'alias d'index à la vérité publiée.
- Migrer métadonnées et objets avec hashes ; qualifier isolation Python, droits d'outils, formats et suppression des artefacts avant transfert.

Écarts et décisions encore ouverts

- Compatibilité complète des outils Python, documents et formats à mesurer ; isolation non qualifiée ici.
- La profondeur de ce lot final est ciblée et ne remplace pas un audit ligne par ligne de chaque outil.
- Qualifier les dépendances des images Milvus et du bus pour ne conserver aucune JVM.

Références de l'analyse

[monorepo](#) · [services/ai-gen-assistant/app/src/context/application/router/context_router.py:103](#) Création et modèle de configuration.

[monorepo](#) · [services/ai-gen-assistant/app/src/context/application/router/context_version_router.py:148](#) Comparaison des versions.

[monorepo](#) · [services/ai-gen-assistant/app/src/assistant/application/router/conversation_router.py:207](#) Création et streaming.

[monorepo](#) · [services/ai-gen-assistant/app/src/corpus/application/router/corpus_router.py:111](#) Surface de corpus et contrôle projet.

[monorepo](#) · [services/ai-gen-assistant/app/src/corpus/infra/adapter/storage/chunk_adapter.py:308](#) Ordre écriture/nettoyage et absence de transaction.

[monorepo](#) · [services/ai-gen-assistant/app/src/assistant/application/router/toolbox_router.py:211](#) Variables de toolbox utilisateur.